SJSU CMPE 172

CMPE 172-02 – MILESTONE 3 REPORT

Online Appointment Scheduling System: Interview Scheduler

Name: Charlie Banh
Student ID: 015345473
Date: October 27, 2026
Chosen scenario: Interview Scheduler to help with resume reviews and mock interviews for software engineers
GitHub repository (milestone tag): https://github.com/c-banh-mi/cmpe172-interview-scheduler/tree/milestone-3
Code-walkthrough video:https://drive.google.com/file/d/1OmLJ7df6l5PLxmGjS5zRz7_moh1DLNOu/view?usp=sharing

1. Overview

Milestone 3 adds a distribution boundary between the scheduler and a mock external notification service. The scheduler reaches it through a Gateway interface that sends one coarse-grained DTO per call. Booking and cancellation commit first. The confirmation is then sent on a background thread with a timeout, retry with exponential backoff, and a fallback table that a scheduled job redelivers from, so a slow or dead notification service never fails a booking. The milestone also adds a logging plan, a GET /health endpoint with a database check, a booking timer and notification counter, and packages the app with a multi-stage Dockerfile, docker-compose, and a GitHub Actions workflow that runs all 79 tests on every push.

Requirement	Where it is implemented
Mock notification service (second controller)	`mocknotify/MockNotificationController`
Notification client (Gateway) + DTO	`notification/NotificationGateway, RestNotificationGateway, NotificationRequest`
Timeout, retry, fallback	`RestNotificationGateway, NotificationSender, pending_notifications`
Commit first, then notify; idempotent retries	`SlotBooker, AppointmentService, AppointmentNotificationListener`
Confirmation on book and cancel	`BOOKING_CONFIRMED / BOOKING_CANCELLED events`
Logging plan	Section 3.1
GET /health with database check	`controller/HealthController`
Metric	`scheduler.booking timer, scheduler.notifications counter`
Failure scenario + recovery	`NotificationRedeliveryJob; Section 3.4`
Dockerfile, docker-compose.yml, GitHub Actions	`Dockerfile, docker-compose.yml, .github/workflows/ci.yml`
Integration test: booking succeeds when the stub is down	`NotificationDownIntegrationTest`

Table 1: Milestone 3 requirements and where they are in the code

2. Distribution Boundary Design

2.1 Distribution diagram

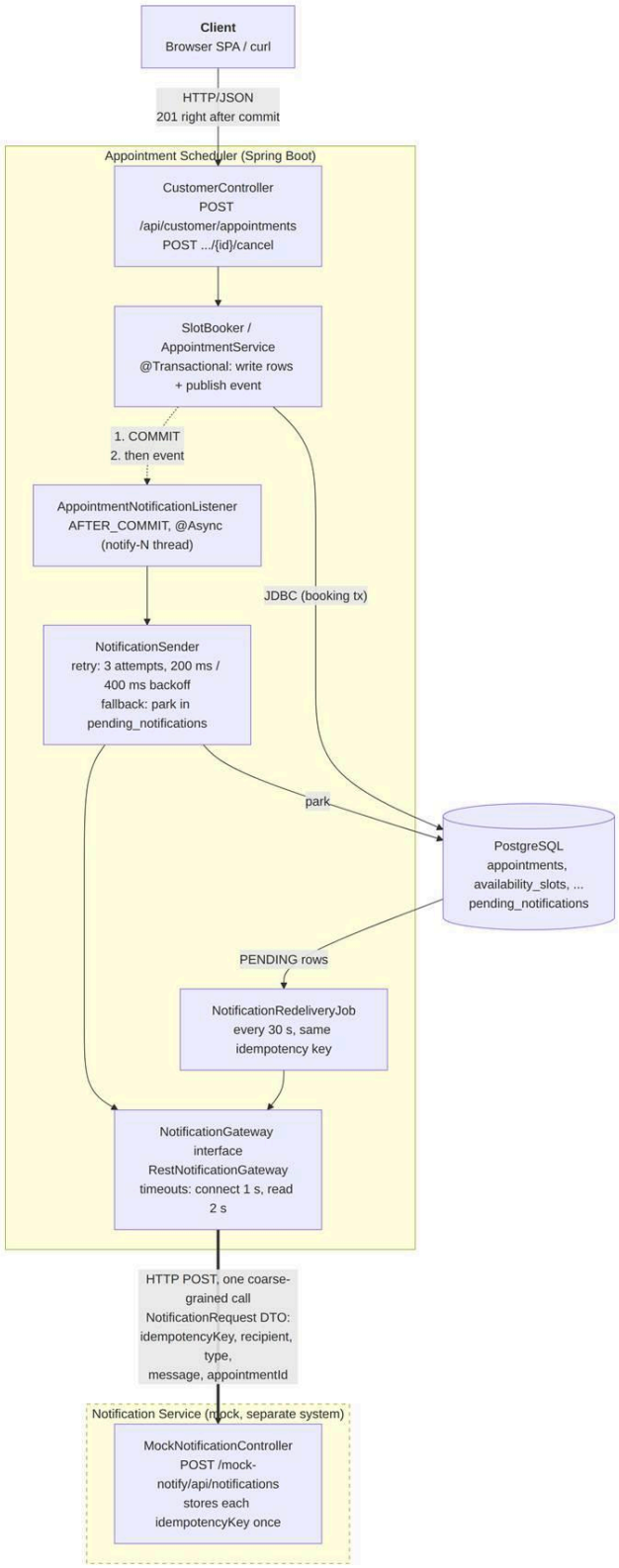


Figure 1: Distribution diagram: Client → Appointment Scheduler → Notification Service (docs/distribution-diagram.png)

The client sends POST /api/customer/appointments to the scheduler. The HTTP request thread runs the whole booking transaction:

1. SlotBooker claims the slot with the version check.
2. It inserts the appointment.
3. It publishes an AppointmentChangedEvent.

Nothing leaves the process yet. Spring holds the event until the transaction commits, and the 201 goes back to the client.

After the commit, AppointmentNotificationListener picks up the event on a notify-N thread. It reads the appointment and the customer's email, builds the NotificationRequest, and hands it to NotificationSender, which calls the NotificationGateway.

Only one thing crosses the boundary: an HTTP POST to /mock-notify/api/notifications carrying the NotificationRequest as JSON plus the Idempotency-Key header. If that call fails, the request is written to pending_notifications in the scheduler's own database. NotificationRedeliveryJob retries it later on the scheduling thread.

For this milestone the mock runs in the same JVM. Even so, the scheduler reaches it only through HTTP and a configurable base URL, so it behaves like a separate service and can be replaced by one.

2.2 Coarse-grained interface and DTO

The scheduler talks to the notification service only through the NotificationGateway interface. One call sends one NotificationRequest:

Field	Type	Example	Purpose
`idempotencyKey`	String	`appointment-4-BOOKING_CONFIRMED`	Same on every retry; service stores it once
`recipient`	String	`jordan@example.com`	Customer email (users.email)
`type`	enum	`BOOKING_CONFIRMED / BOOKING_CANCELLED`	What happened
`message`	String	Hi Jordan Kim, your Mock Coding Interview with Alice Nguyen on Sat, Nov 14 at 4:00 PM is confirmed (appointment #4).	Finished text, ready to send
`appointmentId`	long	4	Reference only; the service does not look it up

Table 2: NotificationRequest DTO fields

```
/**
 * The scheduler's only view of the external notification system (Gateway pattern).
 * Callers depend on this interface, not on HTTP, so the mock can be swapped for a real
 * channel (Discord/Slack webhook, SMTP) without touching booking code.
 */
public interface NotificationGateway {

      /**
      * Delivers one notification with one remote call.
      *
      * @throws NotificationException if the call failed or timed out
      */
      NotificationReceipt send(NotificationRequest request);
}
```

The Gateway interface (notification/NotificationGateway.java, lines 3-16)

```
/**
 * The one DTO sent across the distribution boundary. Coarse-grained: a single call carries
 * everything the notification service needs (who, what kind, the finished message), so the
 * service never has to call back into the scheduler or its database.
 *
 * @param idempotencyKey same value on every retry of the same notification, e.g.
 *                       "appointment-12-BOOKING_CONFIRMED"; the service stores a key only once
 * @param appointmentId  reference for support/debugging; the service does not look it up
 */
public record NotificationRequest(
      String idempotencyKey,
      String recipient,
      NotificationType type,
      String message,
      long appointmentId) {

      public static String keyFor(long appointmentId, NotificationType type) {
      return "appointment-" + appointmentId + "-" + type;
      }
}
```

The DTO sent across the boundary (notification/NotificationRequest.java, lines 3-22)

Wire format: POST /mock-notify/api/notifications with the DTO as JSON and an Idempotency-Key header. Answers: 201 ACCEPTED, 200 DUPLICATE (key already stored), 503 when the mock is switched DOWN. The mock has its own request class (MockNotificationMessage); the two sides share only the JSON shape.

A fine-grained design would split delivery into several remote calls. For example: register or look up the recipient, pick a template, fill it with appointment details, then send. Each of those is a network round trip, so latency adds up. Each is also a separate point of failure. If the third call times out, the service may be left with a half-built notification, and the scheduler has to work out which steps happened. It also couples the two sides tightly, because the scheduler would need to know the service's internal steps and their order.

With one coarse-grained call, a notification is a single unit that was either accepted or not. That is what keeps retry and idempotency simple: one call, one key, one stored result. The message text is built on the scheduler side because the scheduler owns the data it needs: customer name, service, mentor and time. If the notification service built the message, it would have to call back into the scheduler or read its database. That kind of two-way dependency is exactly what a distribution boundary should avoid. The service only receives finished text and delivers it.

NotificationGateway is an interface so that booking code depends on “send one notification,” not on HTTP or the mock. RestNotificationGateway is the only class that knows about URLs, timeouts and status codes. Replacing the mock with a Discord/Slack webhook or SMTP means writing a new implementation without touching SlotBooker or AppointmentService. The Milestone 4 AI-generated reminders can reuse the same gateway and DTO with a new NotificationType.

2.3 Fault handling: timeout, retry, fallback

Mechanism	Setting (application.yml)	Value	Code
Connect timeout	`notification.connect-timeout`	1 s	`RestNotificationGateway`
Read timeout	`notification.read-timeout`	2 s	`RestNotificationGateway`
Retry	`notification.max-attempts / backoff`	3 attempts, wait 200 ms then 400 ms	`NotificationSender.deliver`
What is retried	–	timeouts, refused connections, 5xx; not 4xx	`NotificationException.retryable()`
Fallback	–	store in pending_notifications (PENDING; FAILED for 4xx)	`NotificationSender.fallBack`
Recovery	`notification.redelivery-interval`	every 30 s, one attempt per row	`NotificationRedeliveryJob`
Give up	`notification.max-redeliveries`	10 total attempts → FAILED	`NotificationRedeliveryJob`

Table 3: Fault-handling settings

```
public RestNotificationGateway(RestClient.Builder builder, NotificationProperties props, Environment env) {
    SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
    factory.setConnectTimeout(props.connectTimeout());
    factory.setReadTimeout(props.readTimeout());
      this.http = builder.requestFactory(factory).build();
      this.env = env;
}

@Override
public NotificationReceipt send(NotificationRequest request) {
      try {
      NotificationReceipt receipt = http.post()
                .uri(baseUrl() + "/api/notifications")
                .header("Idempotency-Key", request.idempotencyKey())
                .contentType(MediaType.APPLICATION_JSON)
                .body(request)
              .retrieve()
                .body(NotificationReceipt.class);
      // A 2xx answer means the service accepted it, even if it sent no body.
      return receipt != null ? receipt : new NotificationReceipt(null, "ACCEPTED", null);
      } catch (ResourceAccessException e) {
      // Connection refused, unknown host, or a connect/read timeout.
      throw new NotificationException("Notification service unreachable: " + e.getMessage(), true, e);
      } catch (HttpClientErrorException e) {
      throw new NotificationException("Notification service rejected the request: " + e.getStatusCode(), false,
e);
      } catch (RestClientResponseException e) {
      throw new NotificationException("Notification service error: " + e.getStatusCode(), true, e);
      } catch (RestClientException e) {
      // A read timeout can also surface here, while the answer is being read.
      if (causedByTimeout(e)) {
              throw new NotificationException("Notification service timed out", true, e);
      }
      throw new NotificationException("Notification call failed: " + e.getMessage(), true, e);
      }
}
```

Timeouts and how each failure is classified as retryable or not (notification/RestNotificationGateway.java, lines 26-60)

```
/** Try up to maxAttempts times; if all fail, park the notification for the redelivery job. */
public void deliver(NotificationRequest req) {
      NotificationException last = null;
      int attempts = 0;
      while (attempts < props.maxAttempts()) {
      attempts++;
      try {
            NotificationReceipt receipt = gateway.send(req);
            log.info("Sent {} to {} (key {}, notification {}, attempt {})",
                    req.type(), req.recipient(), req.idempotencyKey(), receipt.notificationId(), attempts);
            count("sent", req);
              return;
      } catch (NotificationException e) {
              last = e;
              if (!e.retryable() || attempts == props.maxAttempts()) {
              break;   // a 4xx will never succeed; or we are out of attempts
              }
              long waitMs = props.backoff().toMillis() << (attempts - 1);   // 200 ms, 400 ms, ...
            log.warn("Notification {} attempt {}/{} failed: {}. Retrying in {} ms",
                    req.idempotencyKey(), attempts, props.maxAttempts(), e.getMessage(), waitMs);
              if (!sleep(waitMs)) {
              break;   // shutting down: park it now instead of waiting
```

```
            }
      }
      }
      fallBack(req, last, attempts);
}

private void fallBack(NotificationRequest req, NotificationException error, int attempts) {
      if (error != null && !error.retryable()) {
      // Resending the same request cannot succeed; keep it for a person to look at.
        log.error("Notification {} rejected, not retrying: {}", req.idempotencyKey(), error.getMessage());
      pending.park(req, PendingNotification.FAILED, attempts, error.getMessage());
        count("failed", req);
      return;
      }
      String reason = error == null ? "no attempt made" : error.getMessage();
    log.warn("Notification service unavailable after {} attempt(s); parked {} for redelivery ({})",
              attempts, req.idempotencyKey(), reason);
      pending.park(req, PendingNotification.PENDING, attempts, reason);
      count("parked", req);
}
```

Retry with exponential backoff, then fallback (notification/NotificationSender.java, lines 31-72)

Notifications never fail a booking. The booking is what the customer cares about, and it has already been committed by the time the notification is sent. Failing the request at that point would show the customer an error for a booking that actually succeeded. A late confirmation is a minor problem; a lost booking or a misleading 500 is a real one. That is why every failure path ends in the fallback table instead of an exception reaching the request.

4xx is not retried. Only failures that might succeed on another try are retried: timeouts, refused connections and 5xx responses, which mean the service is down or overloaded. A 4xx means the service understood the request and rejected it, for example a bad recipient or a malformed body. Sending the identical request again will get the same answer and only adds load. Those requests go straight to FAILED so a person can look at them.

Exponential backoff. Waiting 200 ms and then 400 ms gives a briefly overloaded service time to recover instead of hitting it again immediately. The in-line retries are short on purpose. Anything longer is handed to the redelivery job, which retries every 30 s for up to 10 total attempts, so a background thread is never tied up waiting out a long outage.

2.4 Commit first, then notify

```
// Backstop: the partial unique index uq_appt_active_slot rejects a second BOOKED row
// for this slot even if the check above were ever bypassed (DuplicateKeyException).
long appointmentId = appointments.insert(slot.id(), slot.serviceId(), customerId, req.notes());

// Delivered only after this transaction commits (see AppointmentNotificationListener).
// A lost race rolls back, so a retried booking never sends a second confirmation.
events.publishEvent(new AppointmentChangedEvent(appointmentId, NotificationType.BOOKING_CONFIRMED));
return appointmentId;
```

End of the booking transaction: the event is published after the insert (service/SlotBooker.java, lines 72-79)

```
@Async
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onAppointmentChanged(AppointmentChangedEvent event) {
      if (!props.enabled()) {
      return;
      }
      try {
      AppointmentView appt = appointments.findViewById(event.appointmentId()).orElse(null);
      String email = appointments.findCustomerEmail(event.appointmentId()).orElse(null);
      if (appt == null || email == null) {
            log.warn("Appointment {} vanished before its {} notification was sent",
                    event.appointmentId(), event.type());
              return;
      }
      sender.deliver(new NotificationRequest(
                NotificationRequest.keyFor(appt.id(), event.type()),
              email,
              event.type(),
              message(appt, event.type()),
              appt.id()));
      } catch (RuntimeException e) {
      // Even the fallback failed (e.g. the database is down too). The booking itself is safe.
      log.error("Could not send or park {} notification for appointment {}",
              event.type(), event.appointmentId(), e);
      }
}
```

The listener runs only after commit, on a background thread (notification/AppointmentNotificationListener.java, lines 38-63)

Step	Thread	Inside the DB transaction?
Slot UPDATE (version check) + appointment INSERT	HTTP request thread	Yes
`publishEvent(AppointmentChangedEvent)`	HTTP request thread	Yes (held until commit)
COMMIT, then HTTP 201 to the customer	HTTP request thread	–
Listener builds DTO, gateway call, retries, fallback	`notify-N`	No

Table 4: Order of events for one booking

- Measured: with the notification service DOWN the booking answered in 65 ms; with it SLOW (5 s), the booking still answered at once while the background thread timed out 3 times.

There are two reasons the remote call runs outside the transaction:

1. It would hold locks and connections. A transaction keeps a pooled database connection and the slot's row lock until it commits. If the notification call ran inside it and the service hit the 2 s read timeout three times, that booking would hold both for several seconds. Other bookings for the same slot would wait on the lock, and under load the connection pool would drain.
2. A rollback cannot un-send a message. If the message went out and the transaction then failed (lost race, unique-index violation, commit error), the customer would get a confirmation for a booking that does not exist.

- @TransactionalEventListener(phase = AFTER_COMMIT) handles the second problem. Spring only delivers the event if the publishing transaction commits; if it rolls back, the event is dropped. This fits the Milestone 2 retry loop. An attempt that loses the race throws OptimisticLockingFailureException, its transaction rolls back, and BookingService retries in a new transaction. Even an attempt that got as far as publishing the event sends nothing unless it actually commits. So one committed booking means exactly one confirmation, no matter how many attempts it took.
- @Async moves the listener onto a notify-N thread so the 201 is not delayed by the remote call. That is why the booking was answered in 65 ms with the service down.

2.5 Idempotent retries

- Key per notification: appointment-{id}-{BOOKING_CONFIRMED|BOOKING_CANCELLED}, sent in the body and as the Idempotency-Key header; the same key on every retry and redelivery.
- Mock service stores each key once (computeIfAbsent) and answers DUPLICATE for a repeat.
- Fallback insert is idempotent too: ON CONFLICT (idempotency_key) DO NOTHING.
- A booking attempt that loses the race rolls back, so its event is discarded: one committed booking = one confirmation.

```
public Accepted accept(MockNotificationMessage msg) {
      boolean[] isNew = {false};
      Stored stored = byKey.computeIfAbsent(msg.idempotencyKey(), k -> {
      isNew[0] = true;
      return new Stored(UUID.randomUUID().toString(), msg, Instant.now());
      });
      return new Accepted(stored, !isNew[0]);
}
```

Mock service: one stored copy per idempotency key (mocknotify/MockNotificationStore.java, lines 38-45)

```
WARN [notify-2] NotificationSender: Notification appointment-5-BOOKING_CONFIRMED attempt 1/3 failed: Notification
service timed out. Retrying in 200 ms
WARN [notify-2] NotificationSender: Notification appointment-5-BOOKING_CONFIRMED attempt 2/3 failed: Notification
service timed out. Retrying in 400 ms
INFO [exec-10] MockNotificationController: [mock-notify] Delivered BOOKING_CONFIRMED to jordan@example.com: "Hi
Jordan Kim, ..."
WARN [notify-2] NotificationSender: Notification service unavailable after 3 attempt(s); parked
appointment-5-BOOKING_CONFIRMED for redelivery
INFO [exec-7]  MockNotificationController: [mock-notify] Duplicate appointment-5-BOOKING_CONFIRMED ignored
(already stored as 263e7ad2-...)
INFO [scheduling-1] NotificationRedeliveryJob: Redelivered appointment-5-BOOKING_CONFIRMED to jordan@example.com
after 3 failed attempt(s) (service said DUPLICATE)
```

Real log (Docker, mock SLOW 5 s): Real log (Docker, mock SLOW 5 s): the first request was stored after the scheduler had stopped waiting for it, so the later copies were recognized as DUPLICATE and the customer was notified once.

A timeout only means the scheduler stopped waiting for an answer. It says nothing about whether the service will still process the request. In the SLOW run above, the mock holds each request for 5 s before storing it, well past the 2 s read timeout. From MockNotificationController.answer in SLOW mode:

1. The request arrives at the mock.
2. The mock waits 5 s (Thread.sleep) before doing anything with it.
3. Meanwhile, the scheduler's 2 s read timeout expires. It counts the attempt as failed and retries two more times, then saves the notification for later.
4. After its 5 s wait, the mock finally stores the first request ("Delivered"). The scheduler had already given up on it.
5. The later attempts and the redelivery all arrive with the same key, so the mock answers "Duplicate … ignored".

The idempotency key prevents this. It is derived from the appointment ID and event type, so it is identical on the first attempt, every retry, and the later redelivery. The service stores the first request under that key and answers DUPLICATE for any repeat without delivering again. The scheduler treats DUPLICATE as success.

The log shows exactly this: one “Delivered” line, then DUPLICATE for the later attempts. The scheduler side applies the same idea: ON CONFLICT (idempotency_key) DO NOTHING keeps a notification from being parked twice.

2.6 Confirmation on book and cancel

Action	Where the event is published	Type	Message ends with
Book	`SlotBooker.bookOnce`	`BOOKING_CONFIRMED`	… is confirmed (appointment #N)
Cancel	`AppointmentService.cancel`	`BOOKING_CANCELLED`	… has been cancelled (appointment #N)

Table 5: Notification triggers

Both events go through the same listener, sender and gateway, so cancellations get the same timeout, retry, fallback and idempotency handling as bookings. The keys include the event type, so a booking’s confirmation and its cancellation are never mistaken for duplicates of each other. In Milestone 4, AI-generated reminders will be another NotificationType sent through this same gateway.

3. System Management and Observability

3.1 Logging plan

The logs record three things: business events (bookings and cancellations that were committed), every failure of the remote notification call along with what the system did about it (retry, park, redeliver, give up), and health-check failures. The goal is that someone reading the log can tell, for any appointment, whether it was booked and whether the customer was notified, without opening the database.

Level	Example message (as emitted)	Class	When
INFO	`Customer jordan.dev booked slot 12 (appointment 4, attempt 1)`	`BookingService`	Booking committed
INFO	`Customer jordan.dev cancelled appointment 4 (slot 12 reopened)`	`AppointmentService`	Cancel committed
INFO	`Sent BOOKING_CONFIRMED to jordan@example.com (key appointment-2-BOOKING_CONFIRMED, notification b9fc…, attempt 1)`	`NotificationSender`	Notification accepted
INFO	`Redelivered appointment-4-BOOKING_CONFIRMED to jordan@example.com after 3 failed attempt(s) (service said ACCEPTED)`	`NotificationRedeliveryJob`	Recovery succeeded
WARN	`Notification appointment-4-BOOKING_CONFIRMED attempt 1/3 failed: Notification service error: 503 SERVICE_UNAVAILABLE. Retrying in 200 ms`	`NotificationSender`	One attempt failed
WARN	`Notification service unavailable after 3 attempt(s); parked appointment-4-BOOKING_CONFIRMED for redelivery (…)`	`NotificationSender`	Fallback used
WARN	`Booking failed: customer sam.dev slot 4: Slot 4 is no longer available`	`BookingService`	Failed booking (409)
ERROR	`Notification appointment-7-BOOKING_CONFIRMED rejected, not retrying: 400 BAD_REQUEST`	`NotificationSender`	4xx from the service
ERROR	`Giving up on notification … after 10 attempts: …`	`NotificationRedeliveryJob`	Redelivery exhausted
ERROR	`Health check: database unreachable: FATAL: terminating connection due to administrator command`	`HealthController`	DB down
ERROR	`Could not send or park BOOKING_CONFIRMED notification for appointment …`	`AppointmentNotificationListener`	Fallback itself failed

Table 6: Log messages the code emits

Background notification threads are named notify-N (spring.task.execution.thread-name-prefix), so the logs show which work ran after the response. Real examples: Section 2.5 or docs/m3-test-output.txt.

3.2 Health endpoint: GET /health

```
@GetMapping("/health")
public ResponseEntity<Map<String, Object>> health() {
      Map<String, Object> database = new LinkedHashMap<>();
      Map<String, Object> notifications = new LinkedHashMap<>();
      boolean dbUp;

      long start = System.nanoTime();
      try {
      jdbc.sql("SELECT 1").query(Integer.class).single();
      dbUp = true;
        database.put("status", "UP");
        database.put("responseTimeMs", (System.nanoTime() - start) / 1_000_000);

      int waiting = pending.countByStatus(PendingNotification.PENDING);
      int failed = pending.countByStatus(PendingNotification.FAILED);
        notifications.put("status", waiting + failed == 0 ? "UP" : "DEGRADED");
        notifications.put("pending", waiting);
        notifications.put("failed", failed);
      } catch (DataAccessException e) {
      dbUp = false;
        log.error("Health check: database unreachable: {}", e.getMostSpecificCause().getMessage());
        database.put("status", "DOWN");
        database.put("error", e.getMostSpecificCause().getClass().getSimpleName());
        notifications.put("status", "UNKNOWN");
      }

      Map<String, Object> body = new LinkedHashMap<>();
    body.put("status", dbUp ? "UP" : "DOWN");
    body.put("timestamp", Instant.now());
    body.put("database", database);
    body.put("notifications", notifications);
      return ResponseEntity.status(dbUp ? HttpStatus.OK : HttpStatus.SERVICE_UNAVAILABLE).body(body);
}
```

GET /health (controller/HealthController.java, lines 37-69)

Situation	HTTP	Response (real output)
All good	200	{"status":"UP","database":{"status":"UP","responseTimeMs":0},"notifications":{"status":"UP","pending":0,"failed":0}}
Notification service down	200	{"status":"UP","database":{"status":"UP","responseTimeMs":0},"notifications":{"status":"DEGRADED","pending":1,"failed":0}}
Database stopped	503	{"status":"DOWN","database":{"status":"DOWN","error":"PSQLException"},"notifications":{"status":"UNKNOWN"}}

Table 7: GET /health responses (timestamp field omitted)

- Database check: SELECT 1 through the connection pool (Hikari connection-timeout 5 s, so a dead database answers 503 in about 4.5 s).
- /health reports DOWN (503) only when the database is unreachable, because then the app cannot book, cancel or even list slots. When only the notification service is failing, bookings still work and notifications are safely queued. So the app reports UP, with notifications DEGRADED and the pending count shown.
- Reporting DOWN in that case would cause harm. Docker would mark the container unhealthy, and a load balancer would stop routing traffic to it. Booking would go offline over a secondary dependency that the fallback already covers.
- Docker's HEALTHCHECK calls /health every 15 s and marks the container unhealthy after three failures. In production, a load balancer or orchestrator (for example, a Kubernetes readiness probe) would use the same 200/503 to decide whether to send traffic to the instance. The DEGRADED status and counts are for operators and monitoring: a pending count that keeps growing means the notification service has been down for a while.

3.3 Metric

Metric	Type	Tags	Gives you	URL
`scheduler.booking`	Timer	`outcome = booked | unavailable | conflict | rejected | error`	Bookings, failed bookings, average booking latency = TOTAL_TIME / COUNT	`/actuator/metrics/scheduler.booking?tag=outcome:booked`
`scheduler.notifications`	Counter	`result = sent | parked | redelivered | failed; type`	How often the remote call fails and recovers	`/actuator/metrics/scheduler.notifications?tag=result:parked`

Table 8: Custom metrics (Micrometer, Spring Boot Actuator)

```
/**
 * Metric: every call is timed into "scheduler.booking", tagged with its outcome
 * (booked, unavailable, conflict, rejected, error). That one timer gives bookings and
 * failed bookings (count per outcome) and average booking latency (totalTime / count).
 */
public AppointmentDto book(AppUser customer, BookingRequest req) {
      Timer.Sample sample = Timer.start(metrics);
      String outcome = "error";
      try {
      AppointmentDto booked = bookWithRetry(customer, req);
      outcome = "booked";
      return booked;
      } catch (SlotUnavailableException e) {
      outcome = "unavailable";
        log.warn("Booking failed: customer {} slot {}: {}", customer.username(), req.slotId(), e.getMessage());
      throw e;
      } catch (ConflictException e) {
      outcome = "conflict";
      throw e;
      } catch (ForbiddenException | NotFoundException | IllegalArgumentException e) {
      outcome = "rejected";
      throw e;
      } finally {
        sample.stop(metrics.timer("scheduler.booking", "outcome", outcome));
      }
}
```

Every booking call is timed and tagged with its outcome (service/BookingService.java, lines 44-69)

```
{"name":"scheduler.booking","baseUnit":"seconds","measurements":[{"statistic":"COUNT","value":2.0},{"statistic":"T
OTAL_TIME","value":0.036268202},{"statistic":"MAX","value":0.0}],"availableTags":[]}
```

GET /actuator/metrics/scheduler.booking?tag=outcome:booked after two demo bookings (average latency 0.036 s / 2 = 18 ms)

The main metric is the scheduler.booking timer. Booking is the system's core operation, and one timer answers all three suggested questions, because every call to BookingService.book is timed and tagged with its outcome:

- Successful bookings: the COUNT for outcome:booked.
- Failed bookings: the sum of COUNT across unavailable, conflict, rejected and error.
- Average booking latency: TOTAL_TIME / COUNT, which was 18 ms in the sample above.
- Bookings per hour: the change in the booked COUNT over an hour. A monitoring system scraping the endpoint, such as Prometheus, would compute this as a rate.
- MAX shows 0.0 in the sample because Micrometer's max covers a rolling time window, which had expired before the request was made.
- scheduler.notifications is the second metric and tracks the distribution boundary specifically. A rising parked count without a matching redelivered count means the notification service is down.

3.4 Failure scenario and recovery

Scenario: the notification service is down while a customer books.

Time (Docker run, 2026-10-26)	What happened
02:22:33	Mock switched DOWN; jordan.dev books slot 12 → 201 in 65 ms (appointment 4)
right after	notify-1: attempts 1/3, 2/3, 3/3 all answered 503; waits 200 ms, then 400 ms
02:22:33.7	Parked appointment-4-BOOKING_CONFIRMED in pending_notifications (PENDING)
02:22:34.1	GET /health → UP, notifications DEGRADED, pending 1
(Enter)	Mock switched UP
02:22:44.6	NotificationRedeliveryJob: Redelivered … after 3 failed attempt(s) (service said ACCEPTED); row → SENT
02:22:45.7	GET /health → UP, notifications UP, pending 0

Table 9: Failure and recovery, from a real run (~/show3.sh option d)

```
@Scheduled(fixedDelayString = "${notification.redelivery-interval:PT30S}",
      initialDelayString = "${notification.redelivery-initial-delay:PT15S}")
public void redeliver() {
      if (!props.enabled()) {
      return;
      }
      for (PendingNotification p : pending.findPending(BATCH)) {
      NotificationRequest req = p.request();
      try {
            NotificationReceipt receipt = gateway.send(req);
            pending.markSent(p.id());
            sender.count("redelivered", req);
            log.info("Redelivered {} to {} after {} failed attempt(s) (service said {})",
                    req.idempotencyKey(), req.recipient(), p.attempts(), receipt.status());
      } catch (NotificationException e) {
             boolean giveUp = !e.retryable() || p.attempts() + 1 >= props.maxRedeliveries();
            pending.recordFailure(p.id(), giveUp ? PendingNotification.FAILED : PendingNotification.PENDING,
                    e.getMessage());
              if (giveUp) {
                sender.count("failed", req);
                log.error("Giving up on notification {} after {} attempts: {}",
                        req.idempotencyKey(), p.attempts() + 1, e.getMessage());
              } else {
                log.warn("Redelivery of {} failed (attempt {}): {}",
                        req.idempotencyKey(), p.attempts() + 1, e.getMessage());
              // The service is still down; leave the rest for the next run.
              return;
              }
      }
      }
}
```

Recovery: redeliver parked notifications (notification/NotificationRedeliveryJob.java, lines 33-63)

- Failure: the notification service is unavailable (stopped, returning 503, or too slow to answer within the timeout) while customers are booking and cancelling.
- User impact: none for the core operation. The booking commits and the customer gets the 201 in normal time (65 ms in the run above), because the notification is sent after commit on a background thread. The only effect is that the confirmation arrives late, after the service comes back.
- Detection: three signals appear within a second:
    - NotificationSender logs a WARN for each failed attempt and another when it parks the notification.
    - GET /health switches notifications to DEGRADED and shows the pending count.
    - The scheduler.notifications counter for result=parked increases.

Any one of these tells an operator that the boundary is failing while the app itself is healthy.

- Recovery: automatic. The notification is stored in pending_notifications in the scheduler's own database, so it survives an app restart. NotificationRedeliveryJob runs every 30 s, makes one attempt per pending row, and marks the row SENT on success. In the run above, the confirmation was delivered on the job's next run after the mock was switched back UP, and /health returned to UP with 0 pending.
- If the service is still down, the job stops after the first failure instead of retrying the whole batch, and waits for its next run. Redelivery uses the same idempotency key, so a notification that actually arrived during the outage is answered DUPLICATE rather than sent twice. After 10 total attempts, or on a 4xx, the row becomes FAILED. That is logged at ERROR and counted in /health so a person can follow up.

- Second scenario (database down): stopping the db container makes /health return 503 (Table 7); after docker compose start db the pool reconnects and it returns 200 with no restart.

4. Delivery: Docker and CI

```
# ---- build stage ----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q package -DskipTests

# ---- runtime stage ----
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/interview-scheduler-*.jar app.jar
EXPOSE 8080
# Docker marks the container unhealthy if GET /health stops answering 200 (e.g. database unreachable).
HEALTHCHECK --interval=15s --timeout=5s --start-period=40s --retries=3 \
  CMD curl -fsS http://localhost:8080/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Dockerfile (multi-stage build, health check) (Dockerfile, lines 1-17)

```
  app:
      build: .
      depends_on:
      db:
      condition: service_healthy
      environment:
      DB_URL: jdbc:postgresql://db:5432/scheduler
      # Match the developer's local date. The JVM time zone becomes the DB session
      # time zone, which CURRENT_DATE in seed.sql and the date filter depend on.
      TZ: America/Los_Angeles
      DB_USERNAME: ${DB_USERNAME:-scheduler}
      DB_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD in .env (copy .env.example)}
      # Notification service the gateway calls. Default: the mock controller inside this container
      # (http://localhost:8080/mock-notify). Set it to an unreachable URL to watch the fallback.
      NOTIFICATION_BASE_URL: ${NOTIFICATION_BASE_URL:-http://localhost:8080/mock-notify}
      ports:
      - "8080:8080"
      restart: unless-stopped

volumes:
  pgdata:
```

docker-compose.yml: the app service (PostgreSQL service above it) (docker-compose.yml, lines 18-38)

```
name: CI

on:
  push:
  pull_request:

jobs:
  build:
      runs-on: ubuntu-latest
      steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
      with:
      distribution: temurin
      java-version: '21'
```

```
      cache: maven
      # Integration tests start PostgreSQL via Testcontainers (Docker is preinstalled on ubuntu-latest).
      - name: Build and test
      run: mvn -B verify
      - name: Upload test reports
      if: always()
      uses: actions/upload-artifact@v4
      with:
      name: surefire-reports
      path: target/surefire-reports/
      # Proves the Dockerfile still builds (the image is not pushed anywhere).
      - name: Build Docker image
      run: docker build -t interview-scheduler:ci .
```

GitHub Actions: build and test on every push (.github/workflows/ci.yml, lines 1-28)

CI result: commits `38b7e29` and `5ff3ed9` both **completed / success** (Actions tab of the repository).

5. Testing

Test class	Kind	What it proves
`NotificationDownIntegrationTest`	Integration (PostgreSQL)	Gateway points at a port where nothing listens: booking (201) and cancel (200) succeed, both notifications are parked after 3 attempts, /health is UP + DEGRADED
`NotificationIntegrationTest`	Integration, real HTTP (random port)	One notification per book and per cancel; SLOW service → timeout → parked → redelivered with no duplicate; 503 → parked → redelivered; /health; metric endpoint
`NotificationSenderTest`	Unit	Success first try; transient failure retried then succeeds; down after 3 attempts → PENDING; 4xx → not retried, FAILED
`BookingServiceTest`	Unit	Successful booking publishes exactly one event and is timed; failed booking publishes nothing and counts as unavailable

Table 10: Milestone 3 tests

```
/**
 * Milestone 3 requirement: a booking still succeeds when the notification service is down.
 * "Down" here is real: the gateway points at a port where nothing listens, so every call
 * gets "connection refused" (no mocks involved).
 */
@TestPropertySource(properties = {
        "notification.enabled=true",
        "notification.base-url=http://127.0.0.1:1/mock-notify",
        "notification.backoff=10ms",
        "notification.redelivery-interval=PT1H",
        "notification.redelivery-initial-delay=PT1H"   // tests call redeliver() themselves
})
class NotificationDownIntegrationTest extends IntegrationTestBase {

      @Autowired
      ObjectMapper json;

      @Autowired
      MeterRegistry metrics;

      private String pendingStatus(long appointmentId, String type) {
      return jdbc.sql("""
                        SELECT status FROM pending_notifications
                         WHERE appointment_id = :id AND type = :type
                        """)
                .param("id", appointmentId).param("type", type)
                .query(String.class).optional().orElse(null);
      }

      @Test
      void bookingAndCancelSucceedWhileNotificationServiceIsDown() throws Exception {
      long slotId = openSlotId("alice.mentor", "Mock Coding Interview");
      double parkedBefore = metrics.counter("scheduler.notifications",
                "result", "parked", "type", "BOOKING_CONFIRMED").count();

      // Book: 201 and the appointment is really in the database.
      String body = mvc.perform(post("/api/customer/appointments").with(as("jordan.dev")).with(csrf())
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("{\"slotId\": %d, \"serviceId\": %d}".formatted(slotId,
serviceIdOfSlot(slotId))))
                .andExpect(status().isCreated())
                .andExpect(jsonPath("$.status").value("BOOKED"))
                .andReturn().getResponse().getContentAsString();
      long apptId = json.readTree(body).get("id").asLong();
        mvc.perform(get("/api/customer/appointments").with(as("jordan.dev")))
                .andExpect(jsonPath("$[0].id").value(apptId));

      // After the retries fail in the background, the confirmation is parked for redelivery.
        await().atMost(Duration.ofSeconds(10))
              .until(() -> "PENDING".equals(pendingStatus(apptId, "BOOKING_CONFIRMED")));
        assertThat(metrics.counter("scheduler.notifications",
                "result", "parked", "type", "BOOKING_CONFIRMED").count()).isEqualTo(parkedBefore + 1);
      Integer attempts = jdbc.sql("SELECT attempts FROM pending_notifications WHERE appointment_id = :id")
                .param("id", apptId).query(Integer.class).single();
        assertThat(attempts).isEqualTo(3);

      // Cancel works too, and its notification is parked the same way.
        mvc.perform(post("/api/customer/appointments/{id}/cancel", apptId).with(as("jordan.dev")).with(csrf()))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.status").value("CANCELLED"));
```

```
        await().atMost(Duration.ofSeconds(10))
              .until(() -> "PENDING".equals(pendingStatus(apptId, "BOOKING_CANCELLED")));

      // The app reports itself UP (bookings work) with notifications DEGRADED.
        mvc.perform(get("/health"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.status").value("UP"))
                .andExpect(jsonPath("$.database.status").value("UP"))
                .andExpect(jsonPath("$.notifications.status").value("DEGRADED"))
                .andExpect(jsonPath("$.notifications.pending").value(2));
      }
}
```

Required test: a booking still succeeds when the notification service is down (test/NotificationDownIntegrationTest.java, lines 20-90)

```
Tests run: 79, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

mvn -B verify, PostgreSQL 16 via Testcontainers (full output: docs/m3-test-output.txt)

The test makes the service really unavailable rather than mocking the gateway. notification.base-url points at 127.0.0.1 port 1, where nothing listens. The real RestNotificationGateway therefore makes a real HTTP call and gets “connection refused,” which runs through the same exception classification, retry and fallback code as a production outage. Backoff is shortened to 10 ms to keep the test fast. The redelivery job is pushed out to an hour so it cannot run mid-test and change the rows.

The assertions check, in order:

1. The booking is really committed. The booking returns 201 with status BOOKED, and the appointment appears in the customer’s list.
2. Retries and the metric are correct. After the background retries, the confirmation is in pending_notifications as PENDING with attempts = 3, and the parked counter went up by exactly one.
3. Cancel is covered too. The cancel returns 200 with CANCELLED, and its notification is parked the same way. This covers the requirement for both triggers.
4. Health reports the failure honestly. /health returns 200 with status UP, database UP, notifications DEGRADED and pending = 2. The app reports the failure without claiming to be down.
5. The await() calls allow up to 10 s because parking happens on a notify thread after the HTTP response has already returned.

6. Known limitations

- If the JVM dies after a commit but before the notification is sent or parked, that notification is lost (a transactional outbox would close this gap). Normal shutdown waits up to 15 s for queued notifications.
- Book and cancel notifications run on separate threads; a cancel within milliseconds of booking could arrive first.
- /mock-notify/** and /actuator/metrics/** are open without login for the demo.
- The mock's duplicate check is not durable. The mock keeps idempotency keys in memory, so the check resets when the app restarts. A real service would need a durable key store.
- Metrics reset on restart. Both metrics live in memory, and nothing scrapes or stores them yet (no Prometheus/Grafana).
- Failures between separate machines are not tested. The required test (Section 5) uses a real refused connection: nothing listens on port 1. However, the mock always runs in the same JVM and container as the scheduler. Failures that only happen across a real network are not exercised. Examples include DNS errors, packet loss, a network partition, or a connection that drops partway through a response.
- Multiple instances could duplicate redelivery work. If more than one app instance ran, two redelivery jobs could pick up the same row. The idempotency key makes this safe but wasteful.

Appendix: Test Output

Test class	Tests	Failures	Errors	Time
`SlotServiceTest`	2	0	0	0.959 s
`NotificationIntegrationTest`	5	0	0	9.077 s
`NotificationSenderTest`	4	0	0	0.211 s
`AppointmentServiceTest`	8	0	0	0.092 s
`NotificationDownIntegrationTest`	1	0	0	8.693 s
`BookingFlowIntegrationTest`	9	0	0	0 s
`AuthIntegrationTest`	9	0	0	0.732 s
`ConcurrentBookingTest`	11	0	0	0.838 s
`ApiIntegrationTest`	7	0	0	0.299 s
`BookingServiceTest`	13	0	0	0.168 s
`ProviderSlotServiceTest`	10	0	0	0.099 s
`Total`	79	0	0	BUILD SUCCESS

Table 11: mvn -B verify on 2026-10-26 (PostgreSQL 16 via Testcontainers); full log with the notification log lines: docs/m3-test-output.txt in the repository